

Experiment 2:

Aim: To demonstrate 2 class confusion matrix using python

ALGORITHM:

Import required libraries

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
```

```
import matplotlib.pyplot as plt
```

Actual class labels

```
y_true = [1, 0, 1, 1, 0, 0, 1, 0, 1, 0]
```

Predicted class labels

```
y_pred = [1, 0, 1, 0, 0, 1, 1, 0, 1, 0]
```

Generate Confusion Matrix

```
cm = confusion_matrix(y_true, y_pred)
```

Display Confusion Matrix values

```
print("Confusion Matrix:")
```

```
print(cm)
```

Extract TN, FP, FN, TP

```
TN = cm[0][0]
```

```
FP = cm[0][1]
```

```
FN = cm[1][0]
```

```
TP = cm[1][1]
```

```
print("\nTrue Negative (TN):", TN)
```

```
print("False Positive (FP):", FP)
```

```
print("False Negative (FN):", FN)
```

```
print("True Positive (TP):", TP)
```

Calculate Accuracy

accuracy = (TP + TN) / (TP + TN + FP + FN)

print("\nAccuracy: {:.2f}%".format(accuracy * 100))

Display the Confusion Matrix graphically

disp = ConfusionMatrixDisplay(confusion_matrix=cm,
display_labels=["Negative", "Positive"])

disp.plot(cmap="Blues")

plt.title("2-Class Confusion Matrix")

plt.show()

```
# Import required libraries
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# Actual class labels
y_true = [1, 0, 1, 1, 0, 0, 1, 0, 1, 0]

# Predicted class labels
y_pred = [1, 0, 1, 0, 0, 1, 1, 0, 1, 0]

# Generate Confusion Matrix
cm = confusion_matrix(y_true, y_pred)

# Display Confusion Matrix values
print("Confusion Matrix:")
print(cm)

# Extract TN, FP, FN, TP
TN = cm[0][0]
FP = cm[0][1]
FN = cm[1][0]
TP = cm[1][1]

print("\nTrue Negative (TN):", TN)
print("False Positive (FP):", FP)
print("False Negative (FN):", FN)
print("True Positive (TP):", TP)

# Calculate Accuracy
accuracy = (TP + TN) / (TP + TN + FP + FN)
print("\nAccuracy: {:.2f}%".format(accuracy * 100))

# Display the Confusion Matrix graphically
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
display_labels=["Negative", "Positive"])

disp.plot(cmap="Blues")
plt.title("2-Class Confusion Matrix")
plt.show()
```

OUTPUT:

Confusion Matrix:

```
[[4 1]
 [1 4]]
```

True Negative (TN): 4

False Positive (FP): 1

False Negative (FN): 1

True Positive (TP): 4

Accuracy: 80.00%

